



ABADDON

Gerador de Senhas Criptograficamente Seguras

O **Abaddon** é um gerador de senhas criptograficamente seguras desenvolvido com foco em alta entropia, resistência a ataques de força bruta e transparência dos métodos utilizados para geração de credenciais.

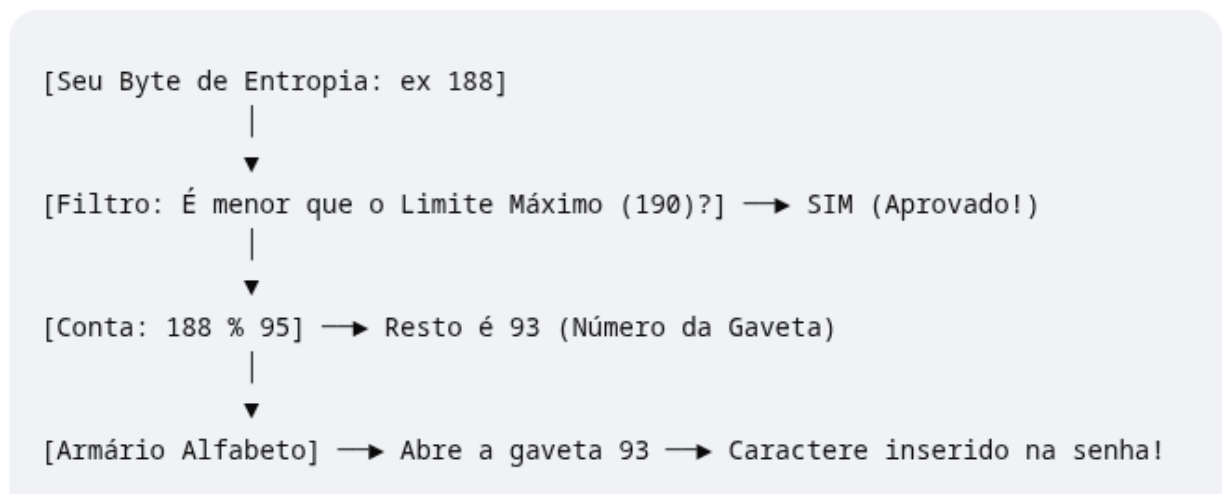
Entropia prevista por caractere:

<i>*Apenas números(10)</i>	<i>3,32 bits</i>
<i>*Letras minúsculas(26)</i>	<i>4,70 bits</i>
<i>*Letras + números(62)</i>	<i>5,95 bits</i>
<i>*ASCII imprimíveis(95)</i>	<i>6,57</i>

Base para Entropia Real Baseada em Silício

O projeto utiliza como fundamento fontes de entropia fornecidas pelo próprio sistema operacional, derivadas de eventos físicos observados pelo hardware do computador. Essas fontes incluem fenômenos eletrônicos e processos imprevisíveis coletados pelo kernel, permitindo a obtenção de números pseudoaleatórios criptograficamente seguros.

O Fluxo Visual do seu Pensamento



Características Principais

- Geração de senhas utilizando fontes de aleatoriedade criptograficamente seguras do sistema operacional.
- Suporte a diferentes conjuntos de caracteres, incluindo números, letras maiúsculas, letras minúsculas e símbolos especiais.
- Cálculo da força teórica da senha com base no universo de caracteres selecionado e no comprimento da senha.
- Implementação voltada para segurança, previsibilidade controlada e redução de vieses estatísticos durante a seleção dos caracteres.

As tomadas de decisões técnicas foram ao longo do projeto mostrando um aprendizado contínuo através dos textos contidos em `./comentarios` na main do código. E se basearam em entropia segura, formulação de teoria da comunicação para achar um logaritmo seguro de base de cálculo para se ter uma materialização da obra apresentada. Foram introduzidas no código:

- * Matemática discreta através de escovação de bits
 - * Teoria da comunicação para encontrar entropia e redundância.
 - * Probabilidade estatística para o rejection sampling
- Visa aprofundamento em teoria dos números para curvas elípticas.

D845CA308DC208185F54BC3107343FA85A8E58EC0FF0B7F8F63009467DA818897FDD41E14124951FA8BB1574BC338B93B7A932FDBF21830BD20D59118038916B86530687BE442E69EC456101699409
25A8CD33842D6B9310DBA55B3890CD98D39324A4466B7AF051D7AE9922D44BE2E4625360B9D20B6B80648965EE3BF999ADCAB0AE54210731773FF6B1729B18EFADE8F271066BC25975B447963628E4
98CA8709A864038682B7AE0E6D58341974DDCA1A9A5BDF4C1CD17E5933AEF033CD9FD5BAF40FAB209C649911CF0BB6DC7E3C158B09B9CABDC58327053C706BB030D1CEFF637BF82F94866BC8575F36
6E2955253DE595007091A7882391B8A63CD2C472057C5EF1FA9D84A18DA513DB68B87B3CCE4037FCE834D9135949EEC39EB0D8C0ADEEF233B9F827D37DA088E3EA125E4DF4E4AC0A6EEEB7BBD6C2E4
C81ADC2BAECF6365EA221A5299D8AE7A51F3A67EAA5AB4D776DE07321A00E604838F225BD57EA2D219F8AC0B6FE54DF3F1B514A8054065B98F37C5466AB4F888007D6B0063E679326839DC9C8DE6A8
884DC33BA5139A46FE8FF010DF5CA9AA522758D27E5E19E601449F7C8715F6B08FA814F5D601E5B79C4DD5C0BDB98F3B095EC9D018E49C1E2F6A11F10AEA2AEE0A2C0CAD9EF5922AD5FEE216B7BE5B
B6B15037136E1649AFEAE6EB42762A99FEE302EF1E748BC2D96BFF863EF62DB26F4ACEA6BF29BEDEF82E697D5C0BE7B08921BF38995BB9744290CD3B3B852E8BB1A57B35C468ED4F28478DF88E748FA
C5FB996AA735C977CB760A92D7DBEBF9B7C44F83D837BF8323036536E26C7169B7757F44BC62B7F33B79FFBAF1ABE28CAF83E03976013324B2A213CE561BEE9DEF2B1E4DE0DAEE909B1DAEB36FC389
139F31328337CB33EA60FAABD50952826E371212ECB82D68E7196E9C3A60E197D1D2D234C471C47CD1EB633B8BB1F5F933B29A1FCC47B3730736035BE51A939647BAB4D964182E9A888EB9AB1D20FC
FF334160B1C5E6C7B14887D7412BA68FBF8675D0B6885A2265E743EA617CEA110B014C00670A8F7B3D58134359F491F318F5B84DD578C0A8EA4A3D5A8A85C3D2C7E97628F192BA97242463C142C5AA
AF0565A99527E10D4F9B1805CEDBA1EC51F86DBD3B0D2123460F2A40E2DF6ABD54A4AE09FDBDE98C873E999C9BB524FA52BDAC8F9D4975AA41792D449A1A9E830F2A30C9964D8DBADB8CAECD0181B4
D5883EFDFAFA1443DEB06733F613443F24648CF51E303A958DF8FCB45130E04D5BA470BA0EFC91C73150F342793D56D683EEA31DCFC848DD3CDE2483C17875245F3F1A556AA7F3E60B9564E901D69C
5084BA3E53057CD7580D96242C26B687D7EBFED2684E3064D826A0D9350A3CF9531A0D0C81BDA33ECFA8663FE814D2C4369D5EF64EEDE6566FCEE9AC609DD1F38AAE25A6714D3B10770596B76E06F2

Escopo e Objetivos do Projeto

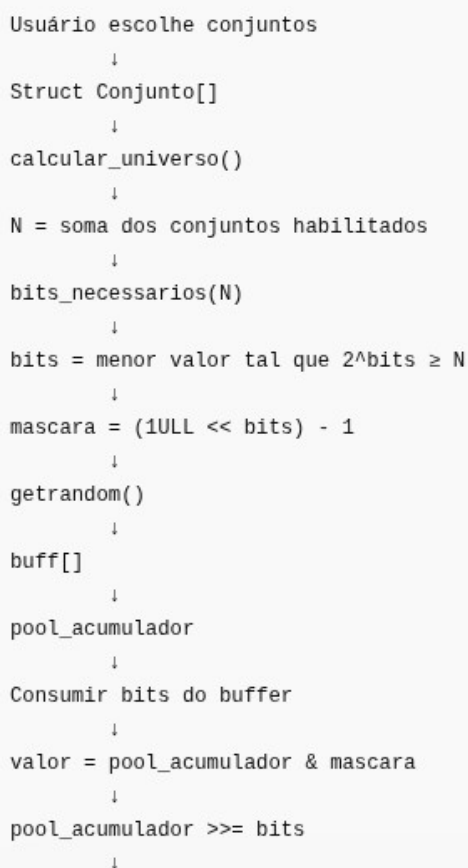
Embora o Abaddon seja capaz de gerar senhas extremamente fortes para uso direto por usuários finais, este não é o foco principal do projeto em seu estado atual. A utilização de senhas longas e de alta entropia normalmente exige o suporte de um gerenciador de senhas para armazenamento e recuperação segura das credenciais, funcionalidade que ainda não faz parte do escopo desta implementação.

Dessa forma, o projeto deve ser compreendido principalmente como uma demonstração prática de geração criptograficamente segura de credenciais, fundamentada em conceitos modernos de entropia, aleatoriedade e teoria da informação.

O principal diferencial do Abaddon está em sua arquitetura simples, modular e facilmente extensível. A lógica de geração foi projetada para separar a origem da

entropia, o cálculo do universo de caracteres e o processo de seleção dos símbolos, permitindo a manutenção e expansão do sistema com relativa facilidade.

Mapa visual de funcionamento:



```
graph TD; A[Usuário escolhe conjuntos] --> B[Struct Conjunto[]]; B --> C[calcular_universo()]; C --> D[N = soma dos conjuntos habilitados]; D --> E[bits_necessarios(N)]; E --> F[bits = menor valor tal que 2^bits ≥ N]; F --> G[mascara = (1ULL << bits) - 1]; G --> H[getrandom()]; H --> I[buff[]]; I --> J[pool_acumulador]; J --> K[Consumir bits do buffer]; K --> L[valor = pool_acumulador & mascara]; L --> M[pool_acumulador >>= bits]; M --> N[...];
```

Usuário escolhe conjuntos
↓
Struct Conjunto[]
↓
calcular_universo()
↓
N = soma dos conjuntos habilitados
↓
bits_necessarios(N)
↓
bits = menor valor tal que $2^{\text{bits}} \geq N$
↓
mascara = (1ULL << bits) - 1
↓
getrandom()
↓
buff[]
↓
pool_acumulador
↓
Consumir bits do buffer
↓
valor = pool_acumulador & mascara
↓
pool_acumulador >>= bits
↓

A struct de conjunto armazena todos os dados referentes aos valores de cada conjunto.

Conjunto é o tipo de caractere escolhido pelo usuário, número, letra, alfanumerico e

etc; Essa struct armazenara dados sobre o comprimento total, o próprio conjunto de universo, o nome e se está habilitado para uso. O universo é a soma do total de caracteres de cada conjunto que pode ser representado por N.

Identificado o tamanho do universo a ser tratado para geração da senha, é passada como parâmetro para função de bits, necessários que aplica uma formula matematica de $2^{(bits)}$ para determinar quantas casas binárias possui o número que será retornado.

A mascara recebera os bits em uma variável unsigned longlong de 64 caracteres possíveis representada em bits -1.

A aleatoriedade foi colocada para ser gerada somente após todas essas etapas do código pois solicita-la no começo fatia com que valores matematicamente sensíveis flutuassem nos processos do computador aguardando decisões do usuário.

A partir desse momento o código segue uma escovação de bits para garantir que cada um seja dividido em 6 bits. E o motivo é o aproveitamento máximo da entropia gerada e maior aproximação das grandezas que serão mostradas mais a frente.

```
↓
Consumir bits do buffer
↓
valor = pool_acumulador & mascara
↓
pool_acumulador >>= bits
↓
Rejection Sampling
(valor >= N ? descarta : continua)
↓
obter_caractere()
↓
Mapeia índice global → caractere real
↓
senha[pos_senha]
↓
Repete até tamanho_senha
↓
'\0'
↓
Senha final
```

Continuando o raciocínio algoritmico, o rejection Sampling é a parte do código que valida se existirá problemas relacionados ao valor gerado. Pois se o usuário escolher um conjunto alfanumérico que contem 62 caracteres, este é o tamanho do universo definido. Mas ao transformar a analise da geração final da senha de byte a byte para bit a bit, $2^{(bits)}$ seria = a 64, então, caso o valor seja maior, o Rejection Sampling é responsável por descartar estes números restantes. Para obter o caractere, o algoritmo faz uma varredura de índice pelo caractere real para aplica-lo no vetor da senha gerada. Este processo se repete até que o tamanho definido pelo usuário seja completamente validado e o laço seja interrompido.

Gerar bytes aleatórios com `getrandom()` e mapear esses bytes para um conjunto explícito de caracteres permitido (charset).

Não usar "ASCII completo" indiscriminadamente.

Por quê?

A fórmula da entropia de uma senha gerada aleatoriamente é:

$$H = x \cdot \log_2(N)$$

onde:

- H = entropia em bits
- x = comprimento da senha
- N = tamanho do conjunto de caracteres possíveis

O que importa é o tamanho efetivo do conjunto de caracteres (N) e a uniformidade da escolha.

Exemplo

- Apenas números: $N = 10$
- Letras minúsculas: $N = 26$
- Maiúsculas + minúsculas + números: $N = 62$
- ASCII imprimível: aproximadamente $N = 95$

Fundamentos

- Utilização de mecanismos modernos de geração de números aleatórios do sistema operacional.
- Entropia derivada de eventos físicos coletados pelo hardware.
- Dependência de fontes reais de aleatoriedade para inicialização do gerador criptográfico.
- Segurança fundamentada em princípios utilizados por sistemas operacionais modernos e aplicações de segurança profissionais.

O nome **Abaddon** faz referência à entidade associada ao abismo e à provação. Da mesma forma, o projeto tem como objetivo criar senhas capazes de resistir aos testes mais rigorosos de segurança, desafiando ataques computacionais através da combinação entre alta entropia, comprimento elevado e geração criptograficamente segura.

* Todo o princípio teórico do projeto se deu pelo estudo da hive-system sobre o poder computacional necessário para se quebrar uma senha. Dentro da pasta imagens no projeto, se concentram de mais variados tipos testes que fizeram e uma extensão busca por fundamentos técnicos para o desenvolvimento do projeto para além de um simples pedaço de código para completar uma task. Porém um trabalho real que mostra o processo correto de se inicializar uma senha segura por este MVP. Não se recomenda ainda a utilização do projeto para fins profissionais pois necessita de revisão técnica e forma adequada de armazenamento segura.

Limitação Técnica

- **Linux 3.17 (2014):** introduziu a syscall `getrandom()` .
- **Linux 5.6 (2020):** houve mudanças importantes no subsistema de aleatoriedade, reduzindo as diferenças práticas entre `/dev/random` e `/dev/urandom` .

Por isso, quando alguém diz que "`/dev/urandom` é seguro em Linux moderno", geralmente está pensando em sistemas com kernels **4.x, 5.x ou 6.x**, que são os encontrados hoje em distribuições amplamente usadas.

Exemplos atuais:

- Ubuntu 22.04, 24.04
- Debian 11, 12
- Fedora recentes
- Arch Linux

- Limitado a sistemas operacionais linux - Versões acima de 3,17

Atualmente o sistema atende exclusivamente a sistemas operacionais linux devido a sua forma de funcionamento e integração com o kernel. O método de captura de aleatoriedade através de uma função de biblioteca externa é mais moderno, mas em contrapartida sacrificou a compatibilidade com kernel de versões inferiores a 3.17.

Além dessa limitação, vou apresentar a outra limitação mostrada e mal interpretada de diversas formas pelos usuários de geradores de senhas.

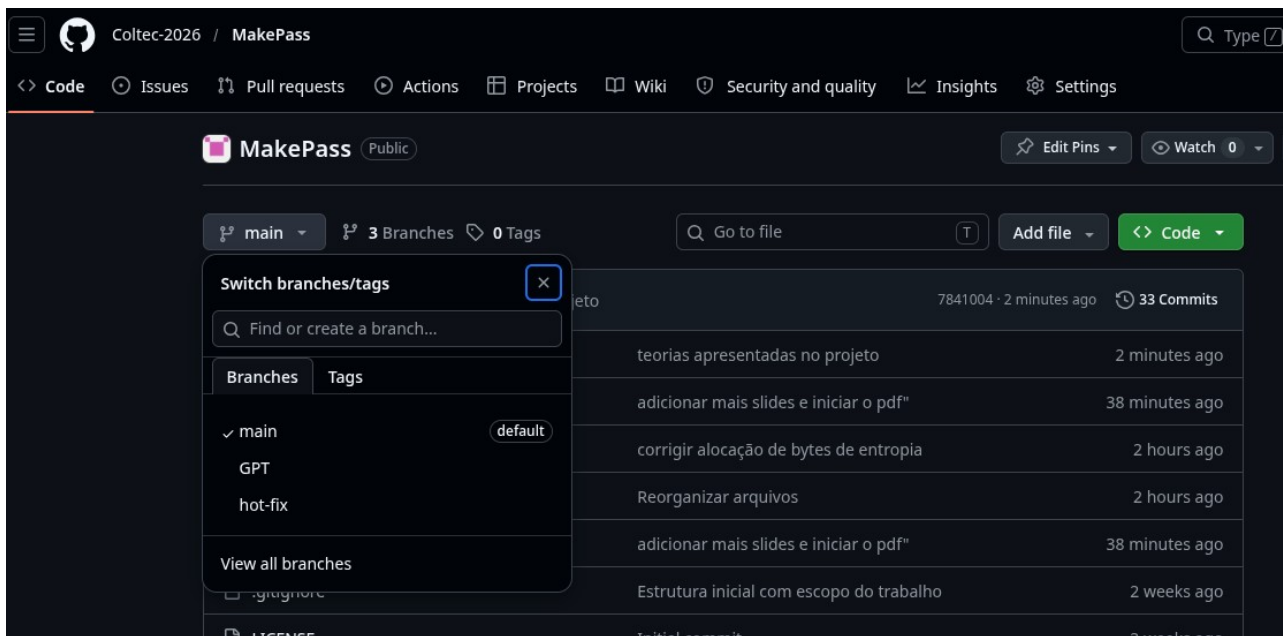
- Cálculo de força de senhas só tem eficácia com entropia segura**
- O software não é imune a reutilização de senhas pelo usuário, use com sabedoria**
- O cálculo de força de entropia, não funciona senhas geradas por humanos**

-

Cálculo de força de senhas padrão $C = N^{(x)}$, medem o número máximo de tentativas para serem quebradas, e não refletem necessariamente o tempo real gasto na quebra da senha. Portanto, não se confunda com números. A sua grandeza se baixa, revela na verdade risco de segurança crítico ou moderado.

- O projeto apresentado é limitado, e apresentado por desenvolvimento contínuo para o ganho de conhecimento. Feito por alguém com limitações técnicas, limitação de tempo e adversidades. Como consequência o código poderá apresentar falhas não percebidas.**

Evolução algorítmica com inclusão de base teorica e prática contínua



Ao longo do projeto foram desenvolvidas 3 branches, essas branches são a própria main, a GPT e a hot-fix.

A branch GPT deu base ao primeiro código gerado por IA que atendia especificamente as tarefas sem expectativas em relação ao resultado esperado, nem atendia aos requisitos propostos para uma senha adequada segundo a teoria da comunicação.

A branch hot-fix, foi feita para que o código saísse as peças funcionando antes do encerramento do prazo. Não sendo considerado boa prática commitar diretamente na master.

```

/* Verifica tamanho válido */
    tamanho = verificarTamanho(tamanho, 16, 512);

    printf("\nDigite 1 para SIM ou 0 para NAO.\n");

/* Escolha dos caracteres */
    do{
        usarMai = verificarDigitacao("Usar letras maiusculas? ", "Erro, digite novamente: ");

        usarMin = verificarDigitacao("Usar letras minusculas? ", "Erro, digite novamente: ");

        usarNum = verificarDigitacao("Usar numeros? ", "Erro, digite novamente: ");

        usarSimb = verificarDigitacao("Usar simbolos? ", "Erro, digite novamente: ");

        if(usarMai == 0 && usarMin == 0 && usarNum == 0 && usarSimb == 0){
            printf("Preencha pelo menos um valor válido! \n ");}
    }while(usarMai == 0 && usarMin == 0 && usarNum == 0 && usarSimb == 0);

/* Monta vetor de caracteres */

```

No código anterior foi incluída diversas funções na tentativa de remover código da main e facilitar o desaboplamento e modularidade. No entanto, mecher em um código produzido por uma máquina é um processo arduo, seja pelas limitações de conhecimento perante a linguagem, seja pelo propósito pelo qual a maquina gerou aquele tipo de código.

```

9      unsigned char buff[1028];
10
11      Conjunto conjuntos[] = {
12          {
13              .nome= "Numeros",
14              .caracteres= "0123456789",
15              .tamanho= 10,
16              .habilitado= 0
17          },
18          {
19              .nome= "Minusculas",
20              .caracteres= "abcdefghijklmnopqrstuvwxyz",
21              .tamanho= 26,
22              .habilitado= 0
23          },
24          {
25              .nome= "Maiusculas",
26              .caracteres= "ABCDEFGHIJKLMNOPQRSTUVWXYZ",
27              .tamanho= 26,
28              .habilitado= 0
29          },
30          {
31              .nome= "Especiais",
32              .caracteres= "!@#$%^&*()-_+=",
33              .tamanho= 15,
34              .habilitado= 0
35          }
36      };
37

```

Perceba ao longo do processo, asdf Foi feito um código a principio muito mais coeso e de forma manual, abandonando completamente a ideia anterior, mais estruturado e fácil de implementar. Embora ainda necessite de refatoração para apresentar coesão em ideias mais complexas. A teoria por trás da implementação, de fato foi algo bem difícil de superar sozinho, foi usado apoio do gemini e do chatgpt para esclarecer situações que somente a leitura e a compreensão sozinha demora dias. Modificando completamente o tipo de uso anterior de tomadora de decisão para agente auxiliar que mostra possibilidades e alternativas, evitando ao máximo o copiar e colar. Infelizmente devido ao tempo, parte do código foi copiado e colado, mas somente após falhar dezenas de vezes e por dificuldades técnicas em relação a tipos primitivos da linguagem, atribuição de bits unicos e tratamento de código de baixo nível combinado com o tempo do projeto vencendo o prazo de entrega. Foi aceita a refatoração feita por IA no trecho descrito a frente:

```
switch (conjunto){  
    case 1:  
        conjuntos[0].habilitado = 1;  
        break;  
    case 2:  
        conjuntos[1].habilitado = 1;  
        break;  
    case 3:  
        conjuntos[0].habilitado = 1;  
        conjuntos[1].habilitado = 1;  
        conjuntos[2].habilitado = 1;  
        break;  
    case 4:  
        conjuntos[0].habilitado = 1;  
        conjuntos[1].habilitado = 1;  
        conjuntos[2].habilitado = 1;  
        conjuntos[3].habilitado = 1;  
        break;  
    default:  
        return 1;  
}
```

Este trecho do código será o responsável pela fácil implementação de diferentes tipos de conjuntos e expansão da ferramenta combinada com as funcionalidade abaixo descritas.

Dificuldades encontradas

O problema do método byte % N

Suponha:

N = 62



Cada caractere precisa representar um dos 62 símbolos possíveis.

A quantidade mínima de bits necessária é:

$$\log_2(62) \approx 5.95$$

Ou seja:

6 bits



já conseguem representar:

0 .. 63



porque:

$$2^6 = 64$$

Uma das principais dificuldades encontradas durante o desenvolvimento do Abaddon esteve relacionada à conversão dos conceitos matemáticos estudados para uma implementação prática em linguagem C. Embora os fundamentos teóricos da entropia e da geração de valores aleatórios sejam relativamente bem documentados, sua aplicação em código exigiu uma compreensão mais profunda sobre representação binária, manipulação de bits e distribuição uniforme de valores.

Inicialmente, a geração dos índices poderia ser realizada através da operação de módulo (%), abordagem amplamente utilizada em implementações simples. Entretanto, verificou-se que esse método introduz viés estatístico quando o tamanho do universo de caracteres não corresponde exatamente a uma potência de dois. Como consequência, alguns caracteres passariam a possuir maior probabilidade de ocorrência do que outros.

Originalmente eu só iria dividir o conjunto pela byte aleatório, porém isso me distanciaria da formula de força aleatória proposta.

Arquitetura usando blocos de bits

Conjuntos

↓

Universo = 62

getrandom()

↓

pool de bits

↓

blocos de 6 bits

↓

índice

↓

caractere



Os conjuntos continuam existindo.

Visualmente

Suponha:

```
buff[0] = 11001010  
buff[1] = 01101101
```



Carrega:

```
11001010
```



Extrai:

```
001010
```



Sobram:

```
11
```



Carrega mais:

```
11 + 01101101
```



Agora possui:

```
110110111
```



para continuar extraindo.

Perceba a complexidade da divisão matemática para aperfeiçoamento da teoria dos conjuntos e da distribuição da entropia gerada.

Vantagens

- Simples
- Rápida
- Fácil de implementar

Desvantagem

Introduz viés quando:

```
256 % N != 0
```



Exemplo:

```
N = 62
```

```
256 % 62 = 8
```



Alguns caracteres aparecem ligeiramente mais vezes.

O método que eu tinha em mente era mais fácil de ser implementado, e como visto nos slides anteriores era concreta. Até que me deparei com essa afirmação sobre a teoria da comunicação. Se a distribuição não for uniforme, Existirá diminuição da aleatoriedade, e este problema afetaria a principal função da ferramenta, ao que foi combatido ao trocar a solução anterior por esta.

Perceba, ter um vício na aleatoriedade, é como jogar cartas com um baralho marcado, ou jogar dados que são viciados.

Diante dessa revelação não só revi os conceitos aplicados no projeto, como percebi a ineficiencia do metodo anterior.

A linha de raciocínio anterior pegava aleatoriedade da função rand(), resetada com srand(), e valor de time nulo. Este por si é um método de baixa entropia matemática. Mas além disso, utilizar uma ferramenta aberta que não seja de criptografia para embaralhar o resultado geraria um vício no resultado. E outro fator que comprometeria a distribuição, seria inserir dígitos em posições definidas antes de embaralhar. Isso é como amassar cartas e jogar no baralho.

2. Aleatoriedade

```
getrandom()  
↓  
buff[]  
↓  
pool_acumulador  
↓  
valor
```



Responsável apenas pela obtenção e consumo da entropia.

Com amadurecimento da percepção, observamos acima a decisão que foi tomada pela aleatoriedade através de um método criptograficamente seguro, armazenada no buffer e depois de dividida em 6 bits os restantes ficam acumuladas dentro de um pool acumulador. Isso faz com que tenha aproveitamento máximo da entropia gerada, pois não é previsível a posição em que será inserido os bits resultantes do resto da operação, melhorando a distribuição da entropia em relação ao método usado e não piorando.

